Kode System Guide — How the Whole Platform Works

Product name: Kode (`kode.com`)
Internal / package names: often still `scanny` (Java packages, Keycloak clients, env keys)
Audience: engineers and operators
Scope: behavior present in the current codebase

---

1. Product overview

Kode is a **QR-powered commerce platform** oriented around Uganda UGX and mobile money. Guests scan a merchant QR, order from their phone, and pay. Merchants run catalog, orders, kitchen, and branch ops. Admins run the platform.

Surface	App	Who	Auth
Merchant + guest web	`src/` · Vite **:5173**	Merchants, branch staff, unauthenticated guests	Keycloak for portal; guests public
Admin console	`admin-console/` · Vite **:5174**	Platform operators	Keycloak `ADMIN`
API	`backend/` · Spring Boot **:4000**	All clients	JWT + public routes

Roles in product language

- **General Manager / owner** — merchant Keycloak login; Overview, Reports, Roles, every branch.
- **Branch Manager** — invited staff (`StaffRole.MANAGER`); day-to-day ops for one branch (orders, kitchen, catalog, floor, venue).
- **Guest / customer** — no login; scans QR; menus and orders are public APIs.
- **Admin** — separate console; merchants, revenue, analytics, health, audit, configs.

Sibling products in the same SPA: **Quick Pay**, **event ticketing**, fullscreen **Kitchen** display (`/kitchen/{businessId}`).

---

2. Repository layout

Path	Role
`src/`	Merchant portal + customer menu + quick pay + tickets + kitchen UI
`admin-console/`	Separate React admin app
`backend/`	Java 21 / Spring Boot 3.4 REST + WebSocket
`docs/`	Architecture, local dev, realtime, security, runbooks
`docker-compose.yml`	Local Postgres 16, Redis 7, Keycloak 26
`start-dev.ps1` / `stop-dev.ps1`	Windows one-shot stack
`tools/bin/kode.ps1`	`kode -All` / `kode -Stop` CLI wrapper

---

3. Local development (`kode -All`)

```
kode -All     # start Keycloak + API (H2) + merchant + admin
kode -Stop    # stop listeners on 4000, 8080, 5173, 5174
```

Service	Port	Notes
Keycloak	**8080**	Admin `admin` / `admin`; then realm bootstrap
API	**4000**	Profile **`h2`**; health `GET /health`
Merchant / customer	**5173**	`--host 0.0.0.0` for LAN phones
Admin	**5174**	Client `scanny-admin`

Phone QR URL often uses LAN IP, e.g. `http://192.168.x.x:5173/b/{businessId}?qr=…` .

H2 vs Postgres

Mode	Database	Schema	Typical use
`kode -All` / profile `h2`	File H2 `./data/scanny`	Hibernate `ddl-auto: update`, **Flyway off**	Fast local Windows
`docker compose` + `dev`	PostgreSQL	**Flyway** migrations	Closer to production

Payments under H2: `fallback-to-stub: true` , stub auto-complete ~10s.

---

4. Authentication (Keycloak)

Realm and clients

Realm: **`scanny`** (bootstrap via `backend/setup-scanny-realm.ps1`).

Client	App	Origin
`scanny-client`	Merchant / customer SPA	`:5173` (+ LAN)
`scanny-admin`	Admin console	`:5174`

Public clients, standard OIDC + PKCE S256. Backend may also use confidential `scanny-backend` in fuller setups.

Realm roles

`MERCHANT` , `ADMIN` , `CUSTOMER` , `STAFF` .

JWT `realm_access.roles` → Spring `ROLE_*` .

Merchant vs staff

- Portal needs `scanny-client` + **`MERCHANT`** **or** **`STAFF`** .
- Pure staff: `STAFF` without `MERCHANT` .

- Backend: `BusinessStaff` by Keycloak subject; branch via `X-Staff-Business`.
- Merchant register: `POST /api/auth/merchant/register` (public).

Local seed logins

Login	Role	Surface
`testuser` / `password`	MERCHANT	:5173
`samantha@scanny.local` / `Samantha@2026!`	MERCHANT	:5173
`adminuser` / `Admin@2026!`	ADMIN	:5174

---

5. Domain model (nitty-gritty)

```
Merchant (Keycloak user, payout MoMo, plan, status)
  └── Business[]  (branch; qrToken; acceptingOrders / busyMode)
        ├── CatalogItem[]
        ├── BusinessTable[] → TableSession[] → TableSessionOrder
        ├── BusinessStaff[]  (email, StaffRole, invite)
        └── Order[]
              ├── OrderLineItem[]
              ├── OrderSplitPayment[]  (split_group_id)
              └── OrderFeedback

PaymentIntent  (ORDER | ORDER_SPLIT | QUICK_PAY | TICKET)
QuickPaymentCode → QuickPaymentTransaction
Ticket → TicketScan
RegisteredDevice / DevicePaymentMethod / DeviceTransaction
QrScanEvent, CookieConsent, AuditEvent, PlatformConfig, OutboxEvent
```

Order money fields

- `subtotal` — merchant MoMo payout base
- `serviceFee`, `psoFee`, `platformFee`
- `merchantPayout` — what merchant receives
- `total` — gross charged to guest

OrderStatus: Pending → Preparing → Ready → Completed | Cancelled
PaymentStatus: Unpaid | Paid | …

Optional: `tableId`, `tableSessionId`, `splitGroupId`, `kitchenNotes`.

StaffRole (DB enum)

`MANAGER`, `CASHIER`, `KITCHEN`, `WAITER`
Invite UX currently emphasizes **Branch Manager** (`MANAGER`) only; others remain as legacy permission maps.

---

6. Guest / customer journey

Entry: `/b/{businessId}?qr=…` → `CustomerApp`.

1. **Open QR URL** — load public menu; record scan.
2. **Menu** — food or lodging (hotel stay) browse.
3. **Cart** — qty, removed ingredients, stay nights; draft in session storage.
4. **Details** — name; table/location (unless stay mode).
5. **Pay** — MTN / Airtel UI choice + phone; optional **split bill**.
6. **Create order** — public `POST` order (fees applied server-side).
7. **Pay full** — `PaymentContext.ORDER`, `referenceId = orderId`.
8. **Or split** — validate shares → create public custom/equal splits → `ORDER_SPLIT` initiate per share with idempotency `split:{splitId}`.
9. **Waiting** — poll status; stub may auto-complete.
10. **Done** — local receipts; track by public id + phone; optional feedback.

Checkout steps: `menu` → `cart` → `details` → `pay` → `waiting` / `done`.

---

7. Merchant portal

View	Purpose
Overview	Branch cards, open / kitchen counts (owner)
Catalog	Items, photos, pricing, lodging
Orders	Live orders; status / payment; split **read-only** panel
Kitchen	Opens `/kitchen/{businessId}`
Roles / Operations	Roles & staff, Venue, Branches
Reports	Owner analytics

Venue settings: accepting orders, busy mode + ETA, pause message, digests.
Branches: create branch; build/sync catalog from Main.

Split on merchant order details

Guests create/pay splits. Merchant sees **read-only**:

- Order total
- **Paid** (sum of Paid shares)
- **Still owed** = total − paid (**knock-off; no refunds**)
- Per-share name, amount, status

Hidden when no shares exist. No merchant “split equally / add share” forms.

---

8. Bill split — internals

Storage

`order_split_payments`: `id`, `splitGroupId`, `orderId`, payer name/phone, `amount`, `paymentStatus`.
Order gets `splitGroupId` when first share is created.

Policy: knock-off, not refund

Paid shares **stay**. They reduce balance owed. No automated refund job.
Order becomes Paid when **all** shares are Paid **and** `paidTotal >= order.total`.

Race safety

On share Paid (webhook / gateway confirm):

1. `SELECT … FOR UPDATE` on parent `Order` (`findByIdForUpdate`)
2. Mark share Paid
3. Recompute; maybe `confirmPaymentFromGateway` on the order

STK idempotency

- Key: `split:{splitId}` (client `Idempotency-Key` + server default for `ORDER_SPLIT`)
- Double-tap reuses Pending / Processing / Paid intent
- Failed / Cancelled clears key so a real retry can start a new STK

Validation (2–8 people)

- Amounts are whole **UGX**
- Equal split: `base = floor(total/n)` , leftover `total % n` adds +1 to first guests
- Custom: names required; phones validated; **sum must equal order total (with fees)**
- Server: `createPublicCustomSplits` enforces the same

Public APIs

Method	Path
GET	`/api/public/orders/{publicId}/splits`
POST	`…/splits/equal`
POST	`…/splits/custom`

`ORDER_SPLIT` payment intents: amount = share amount; **fees not re-applied** on the share (full-order path owns fee math).

---

9. Payments architecture

Contexts

Context	Reference	On Paid
`ORDER`	Order id	Confirm order; mark all splits paid if present
`ORDER_SPLIT`	Split UUID	Confirm share; maybe complete order
`QUICK_PAY`	Tx ref	Complete quick-pay usage
`TICKET`	Purchase ref	Confirm ticket purchase

Fees (defaults)

- Service fee ~ **700 UGX** (`scanny.fees.service-fee-ugx`)
- PSO percent of fee (config) → `psoFee` ; remainder → `platformFee`
- Merchant receives **subtotal** as `merchantPayout`

- Guest pays **subtotal + service fee**

Providers

Id	Status
`stub`	Local default; optional auto-complete
`mtn-momo`	Skeleton — `initiate` / `queryStatus` throw **501**
`airtel-money`	Same skeleton **501**

Aliases: UI `MTN` / `Airtel` → provider ids. Unavailable provider + `fallback-to-stub` → stub.

Flow: initiate → `PaymentIntent` → provider → status / webhook → outbox → paid side effects.

---

10. Floor · Kitchen · Venue

Not separate services — **permission groups / UI surfaces**.

Concept	Meaning
Floor	Tables, table QR links, floor order status, pay-at-table splits. Roles: WAITER, CASHIER, MANAGER. Perms: `floor:tables`, `floor:qr`, `floor:status`.
Kitchen	Kitchen display + advance status. Route `/kitchen/{id}`. Roles: KITCHEN, MANAGER, WAITER. Perms: `kitchen:view`, `kitchen:advance`.
Venue	Accepting orders, busy mode, pause message, settings. Perms: `venue:busy`, `venue:settings`.

---

11. Admin console

Section	Pages
Platform	Overview, Merchants, All Orders, Ticketing, Users
Finance	Revenue & Payments
Analytics	QR Activity, **Cookie Consent**, Reports
System	System Health, Audit Log, Configs

Cookie Consent

- Guest/merchant/admin banners POST `POST /api/consents/cookies`
- Table `cookie_consents` (+ JPA `CookieConsent` for H2 ddl-auto)
- Admin page: trend (Accepted vs Essential), recent events, ranges hourly→yearly
- Weekly lookback uses **days** (`16 * 7`), not `ChronoUnit.WEEKS` on `Instant` (unsupported)

---

12. Realtime

- WebSockets: `/ws/realtime` , `/ws/tickets/stats`
- Pattern: DB commit → publish → Redis pub/sub (when enabled) → WS projection

- Merchant metrics / orders refresh on `QR_SCAN_RECORDED`, `ORDER*`, etc.
- Clients should REST-snapshot on reconnect (no durable replay)

---

13. Ticketing & Quick Pay

Ticketing: create event / classes / tables; public purchase; attendee QR; scans; admin Ticketing page; payment context `TICKET`.

Quick Pay: fixed-price reusable codes; public pay + track; payment context `QUICK_PAY`.

Devices: register device, up to 2 MoMo methods, history APIs.

---

14. Security notes

- Guest menu, order create, public splits, payment initiate/status/webhooks, cookie consent: **public** — treat input as untrusted.
- `/api/admin/**` → `ADMIN`.
- Merchant / staff APIs → JWT + role / `BusinessStaff` checks.
- Several operations HTTP matchers are `permitAll` at the filter; **service-layer** staff/merchant checks apply where wired — hardening area.
- WebSocket auth still being hardened.

---

15. Known incomplete areas

1. Live **MTN / Airtel** collection not implemented (501 stubs).
2. Hosting / production provider not assumed deployed.
3. WhatsApp digests / external messaging mostly flagged off.
4. Legacy staff roles exist; invites focus on Branch Manager.
5. Realtime / some ops paths need tighter auth at the edge.

Appendix — Ports cheat sheet

URL	Use
http://localhost:5173	Merchant + guest + kitchen / tickets
http://localhost:5174	Admin
http://localhost:4000/health	API
http://localhost:4000/h2-console	H2 (h2 profile)
http://localhost:8080	Keycloak
ws://localhost:4000/ws/realtime	Realtime
ws://localhost:4000/ws/tickets/stats	Ticket stats

Internal engineering reference generated from the Kode / scanny codebase.